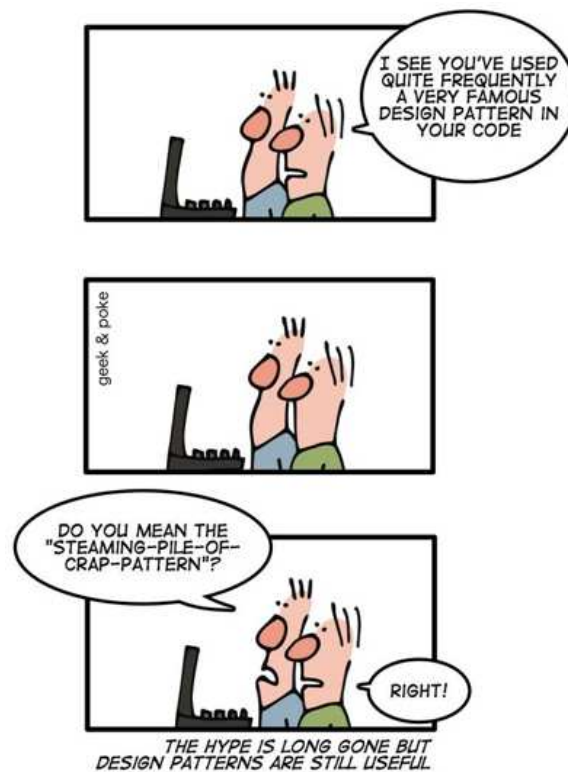

Web Based Requirements Specification & Analysis App

Guidelines and Hints for the development of the project



1 Introduction

This document provides some basic guidelines for the development of the project. We begin with general development tips that should be followed. Then, we provide design directions concerning the project.

2 General Guidelines - DOs and DON'Ts

- Classes
 - ✓ Make **classes small** and cohesive - A single well-defined responsibility for a class
 - ✓ Don't break **encapsulation** by making the data representation public
 - ✓ **Class names** are important – use descriptive names for the concepts represented by the classes
 - ✓ Use **Noun & Noun phrases** for class names
- Methods
 - ✓ Make **methods small** – A method must **do one thing**
 - ✓ **Method names** are important – use descriptive names for the concepts represented by the methods
 - ✓ Use **Verb & Verb phrases** for method names
- Fields
 - ✓ Make **fields private** – A method must do one thing
 - ✓ **Field names** are important – use descriptive names for the concepts represented by the fields
 - ✓ Use **Noun & Noun phrases** for field names
- Follow the standard Java Coding Style

<https://www.oracle.com/java/technologies/javase/codeconventions-contents.html>

3 Application Design and Related Design Patterns

IMPORTANT NOTICE: the design that is given here is a **draft/incomplete** version of a application prototype that relies on the Spring Boot framework. You can use it as a **starting point** and follow it to a certain degree. **Feel free to adapt** what is described in this document to your needs and vision.

To facilitate the maintenance and enable future extensions of the application we assume an architecture that relies on Martin Fowler's catalog of **Enterprise Application Architecture (EAA) patterns** (see <https://martinfowler.com/eaCatalog/>). Maintainability and extensibility are further promoted by the fact that the **Spring Boot framework** heavily relies on the **Model View Controller (MVC) pattern** (see **the lecture slides on Software Design**) for the development of Web applications. MVC allows the clear separation of three different concerns: the **views** of the application that are responsible for the user interaction (UI) with the application, the **domain** model that represents the data handled by the application and the business logic, and the **controllers** that take user input, manipulates the domain model data and updates the view.

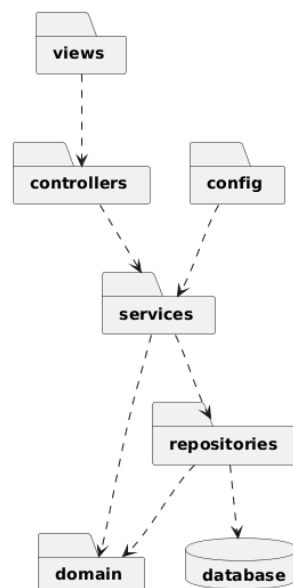


FIGURE 1 APPLICATION ARCHITECTURE.

Figure 1, illustrates a draft architecture for the application.

3.1 Views

The **views package** realizes the user interaction (UI) with the application in collaboration with the **controllers**. Typically, this layer consists of a set of static and dynamic Web pages. The dynamic Web pages are also known as template views (see **Template View pattern** from Martin Fowler's catalog of EAA patterns). A dynamic HTML page renders data from domain model objects (see next) into HTML based on embedded markers. The application controllers are responsible for passing the appropriate domain model objects to the view layer.

3.2 Controllers

The **controllers package** consists of controller classes which take user input from the views of the application, perform certain **service operations** (see next) to manipulate domain model objects, and update the views of the application. The controllers package may comprise at least an AuthController, responsible for the handling user registration and login requests, and a ProjectsController, responsible for handling project management and UML diagram generation requests. The controllers rely on the respective **services** of the services package for serving these requests.

3.3 Services

The **services package** defines a set of services that decouple the business logic of the application from the front-end of the application (controllers and views). Decoupling the business logic from the front-end makes replacing the front-end with another or adding a new front-end easier (e.g. HTML + MVC controllers with React + Rest controllers). Essentially, here we apply the **Service Layer pattern** from Martin Fowler's catalog of EAA patterns.

Indicatively the services package may include a **UserService** that is responsible for the use login, registration and profile management. Moreover, the services package may include a **ProjectsService** that is responsible for managing the user's projects and the generation of UML diagrams.

To cope with the text-based UML tool extensibility requirements of the application you can rely on three well-known GoF design patterns, namely Strategy, Template Method, and Parameterized Factory as discussed below.

The **Strategy pattern** enables the definition of a family of algorithms that can be used interchangeably. In this project, the pattern should be applied to support interchangeable script generation algorithms for different text-based UML tools. First, implement the Strategy pattern for Use Case script generation. Define a common interface representing the Use Case script generation strategy, and provide a separate implementation for each supported text-based UML tool. Apply the same approach to Class Diagram script generation. Define a common interface for Class Diagram script generation strategies and implement one concrete class per text-based UML tool.

The **Template Method pattern** defines the skeleton of an algorithm in a base class and allows subclasses to override specific steps without changing the overall structure. It is useful when developing a family of algorithms that follow similar steps. The base class defines the common workflow so that developers can implement new algorithms consistently and introduce future variations that follow the same structure. The pattern also reduces code duplication by placing the implementation of shared steps in the base class while leaving variable steps to subclasses. In this project, all Use Case script generation algorithms follow the same sequence: generating the script header, defining actors, generating use cases, creating associations between actors and use cases, and producing the script footer. Similarly, all the Class Diagram script generation algorithms follow the same sequence: generating the script header, generating classes (one for each CRC card), defining associations between the classes, and generating the script footer.

The **Parameterized Factory pattern** is a creational design pattern that uses an input parameter within a factory method to determine which class of objects to instantiate, enabling dynamic object creation based on runtime information. Typically, the Parameterized Factory class creates objects that belong to different concrete classes that implement a common interface or derive from common base abstract class. In the project you can create a Parameterized Factory class for the creation of Use Case script generation algorithms and another Parameterized Factory class for the creation of Class Diagram script generation algorithms.

3.4 Repositories

The **repositories package** consists of several interface definitions derived from the general **JpaRepository** interface that is provided by **Spring Boot**. These interfaces allow us to perform basic database operations which map the application data stored in the database to in-memory objects of the domain model classes. Basically, here we apply the **Data Mapper pattern** from **Martin Fowler's catalog of EAA patterns**.

3.5 Domain Model

The backbone of the architecture is the **domain package**. The domain model consists of the basic classes that define the representation of the data handled by the application and the domain logic that is needed for the data manipulation. All the different layers of the application rely on the data model.

The domain model should comprise classes that reflect all the key concepts of the problem domain. Specifically, User is a specific class you should implement for the realization of the user registration and login actions in **Spring Boot**. User should implement the **Spring UserDetails** interface for this reason ¹. **Project, Actor, UseCase, CRCCard** are further concepts that should be in the domain.

The database schema of the application should define corresponding tables for the classes of the application with foreign keys that correspond to the class associations. In Spring Boot the SQL schema can be automatically generated from the domain model with the appropriate JPA annotations ².

4 Tests

Concerning the tests of the application logic you need to develop tests for the different packages of the application (repositories, services, controllers, domain). For the development of the tests we can rely on the SpringBoot testing, JUnit and mocking facilities.

¹ https://github.com/zarras/myy803_springboot_web_app_tutorials/tree/master/sb_tutorial_7_signup_signin

² https://github.com/zarras/myy803_springboot_web_app_tutorials, https://github.com/zarras/myy803_springboot_jpa_tutorials, <https://www.baeldung.com/jpa-many-to-many>